The Journal of Open Source Software

ChemInformant: A Modern, Lightweight Python Client for PubChem with Robust Caching and Validation

Zhiang He[1]

1 Independent researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Evan Spotte-Smith

Reviewers:

- @cthoyt
- @Sulstice
- @LIVazquezS

Submitted: 14 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

ChemInformant provides a **lightweight Pythonic interface** characterized by its ease of use for accessing chemical compound data from the PubChem database (Kim et al., 2016, 2023). Designed to abstract the complexities of the PubChem API (`import ChemInformant as ci`), it facilitates streamlined data retrieval for researchers. Key distinguishing features include **built-in automatic caching** via `requests-cache` (Roman & others, 2024), which enhances performance and contributes to reproducibility; **structured data validation** using Pydantic (Colvin & others, 2025) models (`CompoundData`), promoting data quality; **explicit handling of chemical name ambiguity**, enhancing accuracy; and **optimized batch retrieval**. Furthermore, ChemInformant is built upon a minimal set of **modern, actively maintained core dependencies** (`requests` (Reitz, 2024), `requests-cache`, `Pydantic`), which contributes to its long-term stability and compatibility. This design addresses certain limitations associated with direct API utilization and offers enhancements over existing tools that may be built upon less contemporary or less actively maintained software foundations. ChemInformant is designed as a reliable and efficient tool for routine chemical information retrieval.

Statement of Need

Programmatic access to PubChem (Kim et al., 2023) is integral for numerous chemistry-related research tasks; however, direct utilization of the PUG REST API (Hähnke et al., 2018; Kim et al., 2018) can entail considerable overhead in request handling, response parsing, error management, and data validation. This inherent complexity can impede research progress and potentially introduce errors.

While existing libraries, such as PubChemPy (Swain, 2017), provide an interface to PubChem, `ChemInformant` was developed to address specific limitations and to offer a contemporary, robust, and efficient alternative tailored to common research workflows by focusing on the following aspects:

1. **Dependency Modernity and Maintenance:** `ChemInformant` is designed to rely on a small set of ubiquitous, actively developed, and well-maintained libraries (`requests` (Reitz, 2024), `requests-cache` (Roman & others, 2024), `Pydantic` (Colvin & others, 2025)). This approach supports long-term stability, security, and compatibility with evolving Python (Python Software Foundation, 2025) environments.
2. **Integrated Automatic Caching:** `ChemInformant` integrates `requests-cache` **by default**. This feature provides transparent, persistent caching, which substantially improves performance for repeated queries and batch operations, enhances operational stability during network variability, and contributes to the reproducibility of analyses. The caching is also readily configurable (`ci.setup_cache()`).
3. **Rigorous Data Validation and Structure:** `ChemInformant` employs Pydantic to define a clear `CompoundData` model, ensuring that retrieved data are validated at runtime, data types are enforced, and users interact with consistently structured objects.

4. **Explicit Ambiguity Handling:** `ChemInformant` explicitly addresses chemical name ambiguity by raising a distinct `AmbiguousIdentifierError` containing all potential CIDs, thereby preventing silent failures or arbitrary selections.
5. **Simplicity and Ease of Use:** The API is designed to be Pythonic and intuitive (e.g., `ci.info()`, `ci.cas()`), reducing the adoption barrier for researchers seeking to integrate PubChem data access into Python scripts or Jupyter notebooks (Kluyver et al., 2016).

Through these focused enhancements, `ChemInformant` serves as a reliable, efficient, and developer-friendly tool for routine chemical information retrieval tasks. The target users include cheminformatics researchers, drug discovery scientists, computational chemists, data scientists, and educators.

Key Functionality and Architecture

ChemInformant's capabilities are delivered through a modular architecture (refer to Figure @fig:architecture_flowchart) and are characterized by the following key features:

- **Lightweight & Pythonic Interface:** A concise API accessible via the `ci` alias (e.g., `ci.info()`, `ci.cid()`).
- **Built-in Automatic Caching:** Leverages `requests-cache` (Roman & others, 2024) for transparent caching (default: SQLite, 7-day expiry, caches 404 responses), configurable via `ci.setup_cache()`.
- **Structured Validated Data:** Utilizes a Pydantic (Colvin & others, 2025) `CompoundData` model for data consistency and robustness.
- **Explicit Ambiguity Handling:** Raises `AmbiguousIdentifierError` for ambiguous names and `NotFoundError` for unfound identifiers.
- **Optimized Batch Data Retrieval:** Employs `ci.get_multiple_compounds()` for efficient processing of compound lists.
- **Modern Dependencies:** Built upon `requests` (Reitz, 2024), `requests-cache`, and `Pydantic`.

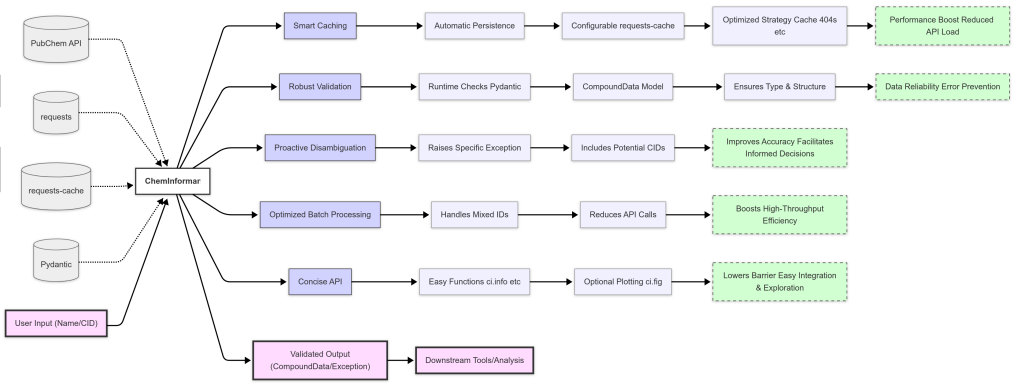

Figure 1: ChemInformant Architecture Diagram

Figure 1: ChemInformant architecture. Inputs (user input, PubChem API, dependencies) flow into the central component. ChemInformant utilizes key features (smart caching, robust validation, proactive disambiguation, optimized batch processing, concise API) to produce validated outputs (`CompoundData` objects or specific exceptions), suitable for downstream tools and analysis.

Example Usage

The following example demonstrates basic usage of ChemInformant:

```python
# Recommended import alias
import ChemInformant as ci

# Optional: Configure cache (e.g., in-memory, 1-hour expiry)
# ci.setup_cache(backend='memory', expire_after=3600)

# Retrieve basic information about Aspirin (by name or CID)
compound = ci.info("Aspirin")
print("Name:", compound.common_name)
print("Formula:", compound.molecular_formula)
print("Weight:", compound.molecular_weight)
print("PubChem URL:", compound.pubchem_url)

# Display 2D structure (requires Pillow and matplotlib)
ci.fig(compound.cid)

# Retrieve CAS number for Ethanol (CID = 702)
ethanol_cas = ci.cas(702)
print("Ethanol CAS:", ethanol_cas)

# Batch retrieval: mix of names and CIDs
ids = ["Water", 2244, "NonExistent", "glucose"]
results = ci.get_multiple_compounds(ids)

print("\nBatch Results:")
for ident, result in results.items():
    if isinstance(result, ci.CompoundData):
        print(ident, "→", result.molecular_formula)
    else:
        print(ident, "→", type(result).__name__)
```

Feature Comparison and Advantages

The features detailed in Table 1, particularly when contrasted with the baseline (PCP v1.0.4), highlight ChemInformant's potential to enhance efficiency, data reliability, and overall robustness in routine cheminformatics workflows. Its design philosophy prioritizes ease of use alongside modern software engineering practices.

Table 1: Comparative analysis of cheminformatics features: ChemInformant (CI) versus PubChemPy (PCP v1.0.4 baseline).

Feature (Cheminformatics Relevance)	ChemInformant (CI)	PubChemPy (PCP v1.0.4)
Automatic Persistent HTTP Caching (Efficiency in Repetitive Data Access)	Implements built-in, transparent HTTP caching via `requests-cache`, configurable (e.g., SQLite backend, 7-day expiry, caches 404 responses). This significantly accelerates repetitive chemical data lookups by minimizing network requests.	Lacks a global persistent HTTP response caching mechanism; primarily relies on instance-level attribute caching for chemical properties. This approach is less effective for performance optimization across sessions or during large dataset processing.

DRAFT

Feature (Cheminformatics Relevance)	ChemInformant (CI)	PubChemPy (PCP v1.0.4)
Runtime Data Validation(Integrity of Chemical Information)	Employs Pydantic models (`CompoundData`) for **strict structural and data type validation** of retrieved chemical data (e.g., SMILES, MW, InChI). This ensures data integrity for subsequent modeling or analysis tasks.	Does not incorporate an explicit client-side data validation layer for chemical attributes. Consequently, there is a potential for inconsistent data types or structures to propagate into downstream processes.
Chemical Name Disambiguation(Accuracy of Identifier Mapping)	**Proactively raises an `AmbiguousIdentifierError`** with candidate CIDs for ambiguous chemical names. This mechanism guides users towards accurate entity selection, crucial for maintaining dataset quality.	Typically returns a list of matching chemical entities, necessitating user-side logic for disambiguation. This may introduce a higher propensity for errors in automated workflows if not explicitly handled.
Optimized Batch Retrieval(Efficiency for Multiple Entities/Properties)	Single call to `ci.get_multiple_compounds(ids)`: supports mixed ID types, automatically handles classification, pagination (200 entries/batch), and rate limiting (5 reqs/sec); error entries are encapsulated as exception objects, while valid entries return complete `CompoundData`.	Requires users to manually split IDs by namespace, then loop through `get_properties` / `get_compounds`, and self-manage request rates (e.g., via `sleep`); if batch size exceeds limits or URL construction is incorrect, the entire batch request fails.
Integrated 2D Chemical Structure Plotting(Rapid Visual Inspection)	Includes a convenience function, `ci.fig()` (requires `matplotlib`, `Pillow`), for direct 2D visualization of chemical structures, facilitating quick data exploration and reporting.	Lacks a built-in, direct display function for chemical structures; provides image URLs that require user-side processing for visualization.
Core Dependency Status(Long-term Sustainability and Ecosystem Integration)	Relies on **modern, actively maintained libraries** (`requests`, `requests-cache`, `Pydantic`), ensuring long-term viability and robust integration with other contemporary cheminformatics tools.	The library (v1.0.4) has not received significant updates since its 2017 release. This raises potential concerns regarding long-term maintenance and compatibility within evolving cheminformatics pipelines.

Acknowledgement

We acknowledge the PubChem database (Kim et al., 2023) as the primary data source. This work utilizes a series of open-source Python libraries for tasks including data requests, caching, validation, as well as optional image processing and data visualization.

Colvin, S., & others. (2025). *Pydantic: Data validation and settings management using*

python type hints. https://doi.org/10.5281/zenodo.15306516

Hähnke, V. D., Kim, S., & Bolton, E. E. (2018). PubChem chemical structure standardization. *Journal of Cheminformatics*, *10*, 36. https://doi.org/10.1186/s13321-018-0293-8

Kim, S., Chen, J., Cheng, T., Gindulyte, A., He, J., He, S., Li, Q., Shoemaker, B. A., Thiessen, P. A., Yu, B., Zaslavsky, L., Zhang, J., & Bolton, E. E. (2023). PubChem 2023 update. *Nucleic Acids Research*, *51*(D1), D1373–D1380. https://doi.org/10.1093/nar/gkac956

Kim, S., Thiessen, P. A., Bolton, E. E., Chen, J., Fu, G., Gindulyte, A., Han, L., He, J., He, S., Shoemaker, B. A., Wang, J., Yu, B., Zhang, J., & Bryant, S. H. (2016). PubChem substance and compound databases. *Nucleic Acids Research*, *44*(D1), D1202–D1213. https://doi.org/10.1093/nar/gkv951

Kim, S., Thiessen, P. A., Cheng, T., Yu, B., & Bolton, E. E. (2018). An update on PUG-REST: RESTful interface for programmatic access to PubChem. *Nucleic Acids Research*, *46*(W1), W563–W570. https://doi.org/10.1093/nar/gky294

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B. E., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J. B., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., & Willing, C. (2016). Jupyter notebooks – a publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

Python Software Foundation. (2025). *Python language reference*. https://www.python.org

Reitz, K. (2024). *Requests: HTTP for humans™*. https://requests.readthedocs.io/

Roman, J. W., & others. (2024). *Requests-cache: Transparent persistent cache for python requests*. https://github.com/requests-cache/requests-cache

Swain, M. (2017). *mcs07/PubChemPy: PubChemPy v1.0.4*. https://doi.org/10.5281/zenodo.541438

DRAFT